TRƯỜNG ĐẠI HỌC QUY NHƠN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP CHƯƠNG 6

4651050279 – Nguyễn Thành Toàn

MẪU THIẾT KẾ COMPOSITE
BÀI TOÁN KẾT HỢP CÁC SHOW
Exercise 1 | Báo cáo thảo luận nhóm

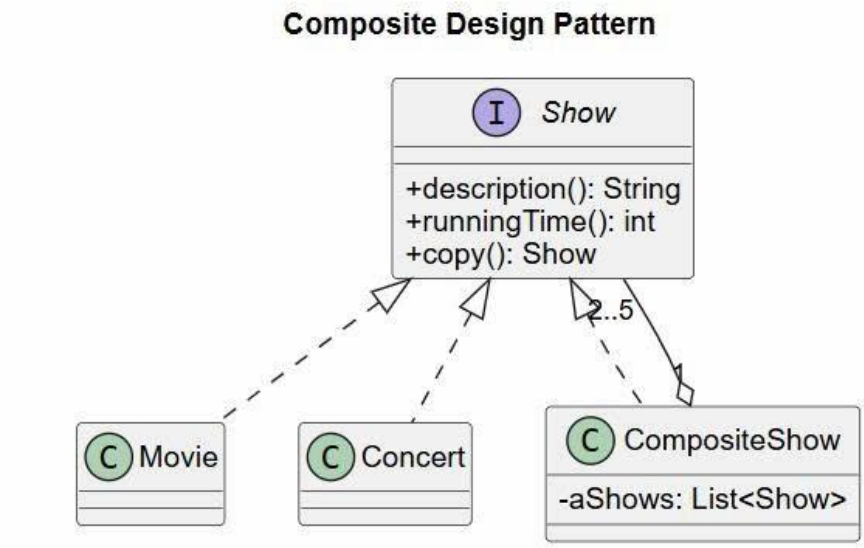
GIA LAI, 2026

1 Yêu cầu bài tập

Trong bài tập này, nhóm chúng em tìm hiểu lời giải sử dụng mẫu thiết kế Composite để kết hợp nhiều show thành một show chung. Nhóm vẽ sơ đồ UML để làm rõ quan hệ giữa các lớp, sau đó đưa ra tình huống sử dụng code và so sánh với cách xử lý khi không dùng Composite.

Đề bài cho sẵn interface Show và hai lớp Movie, Concert. Yêu cầu là xây dựng lớp CompositeShow chứa từ 2 đến 5 show con. Sau đó, tạo một chương trình gồm một concert trước hai movie, trong đó hai movie được gộp thành một đối tượng Show.

2 Sơ đồ UML minh họa của nhóm



Hình 1 Sơ đồ UML nhóm xây dựng để minh họa giải pháp Composite

Nhóm biểu diễn Show là interface chung với ba phương thức: description() trả về mô tả, runningTime() trả về thời lượng và copy() dùng để sao chép show. Movie và Concert là các show đơn lẻ, có vai trò Leaf. CompositeShow là show tổng hợp, có vai trò Composite và lưu các show con trong aShows: List<Show>.

Các đường nét đứt có mũi tên tam giác rỗng hướng về Show cho biết Movie, Concert và CompositeShow đều triển khai interface Show. Vì vậy, các đối tượng của ba lớp này đều có thể được sử dụng qua biến kiểu Show.

Đường nối có hình thoi rỗng ở phía CompositeShow thể hiện quan hệ kết tập: một CompositeShow chứa các Show con. Số 2..5 cho biết mỗi CompositeShow cần có từ 2 đến 5 show con trực tiếp. Một show con cũng có thể là CompositeShow khác, nên có thể tạo các nhóm show lồng nhau.

3 Giải thích mã nguồn

3.1 Tạo show tổng hợp

```
private List<Show> aShows = new ArrayList<>();

public CompositeShow(Show... pShows) {
    if (pShows.length < 2 || pShows.length > 5) {
        throw new IllegalArgumentException(
            "Arguments should be between two and five shows inclusively");
    }
    aShows.addAll(Arrays.asList(pShows));
}
```

Theo cách nhóm chúng em hiểu, Show... cho phép truyền nhiều show vào constructor. Chương trình kiểm tra số lượng show trước. Nếu có ít hơn 2 hoặc nhiều hơn 5 show thì phát sinh IllegalArgumentException; nếu hợp lệ, các show được thêm vào aShows theo thứ tự truyền vào.

Điều kiện 2..5 chỉ tính số show con ngay bên trong mỗi CompositeShow. Ví dụ, một concert và một nhóm hai movie được tính là hai show con của đối tượng ngoài cùng. Danh sách aShows được khai báo private và trong lời giải không có phương thức công khai để thêm hoặc xóa show sau khi tạo.

3.2 Lấy mô tả và tính thời lượng

```
public String description() {
    StringJoiner description = new StringJoiner("; ", "[", "]");
    for (Show show : aShows) {
        description.add(show.description());
    }
    return description.toString();
}

public int runningTime() {
    int time = 0;
    for (Show show : aShows) {
        time += show.runningTime();
    }
    return time;
}
```

Phương thức description() gọi mô tả của từng show con rồi ghép lại, ngăn cách bằng dấu chấm phẩy và đặt trong ngoặc vuông. Phương thức runningTime() cộng thời lượng của các show con. Nếu show con là một nhóm show, nhóm đó sẽ tự tính thời lượng của các phần tử bên trong rồi trả kết quả về.

3.3 Các phương thức còn lại

Phương thức equals() dùng để so sánh hai CompositeShow dựa trên danh sách show con, có xét đến thứ tự. Việc so sánh từng show con phụ thuộc vào equals() của lớp tương ứng. hashCode() cũng được tạo từ aShows để phù hợp với cách so sánh này.

Trong mã được cung cấp, copy() hiện trả về null. Phần này được để lại cho Exercise 10, nên ở bài tập này nhóm chúng em tập trung vào cách kết hợp show, lấy mô tả và tính thời lượng.

4 Trường hợp sử dụng trong chương trình thực tế

4.1 Lập chương trình cho một sự kiện

Giả sử ban tổ chức một sự kiện cần sắp xếp một buổi biểu diễn ca nhạc, sau đó là suất chiếu gồm hai bộ phim. Người phụ trách muốn xem mô tả và tổng thời lượng của cả sự kiện, đồng thời vẫn có thể xem riêng thời lượng của suất chiếu phim. Nhóm sử dụng dữ liệu 100, 120 và 130 phút như trong lời giải để minh họa tình huống này.

```
Show concert = new Concert("Concert 1", "Artist X", 100);
Show movie1 = new Movie("Movie 1", 2017, 120);
Show movie2 = new Movie("Movie 2", 2018, 130);

Show movieSession = new CompositeShow(movie1, movie2);
Show event = new CompositeShow(concert, movieSession);
```

movieSession đại diện cho suất chiếu hai phim. event đại diện cho toàn bộ sự kiện. Mỗi đối tượng CompositeShow chứa hai show con trực tiếp nên đều hợp lệ. List giữ thứ tự concert trước, suất chiếu phim sau; việc phát hoặc trình chiếu thực tế không nằm trong chức năng của đoạn code này.

4.2 Dùng chung một hàm hiển thị thông tin

Trong chương trình quản lý sự kiện, nhóm có thể viết hàm sau bên trong lớp client. Hàm nhận một Show nên dùng được cho một phim, một suất chiếu hoặc cả sự kiện.

```
static void printShowInfo(Show show) {
    System.out.println(show.description());
    System.out.println("Thoi luong: " + show.runningTime() + " phut");
}

// Cac loi goi trong main
printShowInfo(movie1);         // 120 phut
printShowInfo(movieSession);   // 250 phut
printShowInfo(event);          // 350 phut
```

Khi truyền event, runningTime() cộng 100 phút của concert với 250 phút do movieSession trả về. Hàm printShowInfo không cần kiểm tra đối tượng là Movie, Concert hay CompositeShow. Các đoạn ví dụ sử dụng Show, Movie, Concert có sẵn và CompositeShow trong lời giải.

4.3 Tái sử dụng suất chiếu cho chương trình khác

```
Show opening = new Concert("Opening", "Artist Y", 30);
Show anotherEvent = new CompositeShow(opening, movieSession);
printShowInfo(anotherEvent);   // 30 + 250 = 280 phut
```

Nếu chương trình khác cũng có cùng suất chiếu hai phim, nhóm có thể dùng lại movieSession, không cần ghép lại hai phim trong code của client. Đây là việc dùng chung đối tượng trong mô hình phần mềm, không có nghĩa hai sự kiện thực tế diễn ra cùng thời điểm.

5 Nếu không sử dụng Composite thì xử lý thế nào

5.1 Tính trực tiếp cho chương trình đơn giản

Nếu chỉ cần tính thời lượng một sự kiện có ba show cố định, nhóm có thể cộng trực tiếp mà không tạo CompositeShow. Cách này vẫn cho kết quả đúng và phù hợp khi yêu cầu còn đơn giản.

```
int movieTime = movie1.runningTime() + movie2.runningTime();
int eventTime = concert.runningTime() + movieTime;
System.out.println(eventTime);   // 350


String movieDescription = "[" + movie1.description()
    + "; " + movie2.description() + "]";
String eventDescription = "[" + concert.description()
    + "; " + movieDescription + "]";
```

Tuy nhiên, lúc này movieTime chỉ là số nguyên và movieDescription chỉ là chuỗi. Hai movie chưa được biểu diễn bằng một Show chung. Vì vậy, cách viết này chưa đáp ứng yêu cầu gộp hai movie thành một đối tượng Show trong đề bài.

5.2 Khi cần quản lý nhiều chương trình

Nếu không dùng Composite, nhóm vẫn có thể lưu các show trong List<Show> rồi viết vòng lặp để tính tổng. Tuy nhiên, một List không phải là Show, nên không truyền trực tiếp được danh sách đó vào printShowInfo(Show show). Client cần thêm hàm nhận danh sách hoặc một lớp xử lý riêng.

```
List<Show> movies = Arrays.asList(movie1, movie2);
int total = 0;
for (Show movie : movies) {
    total += movie.runningTime();
}
// printShowInfo(movies); // Khong hop le: List<Show> khong la Show
```

Khi có thêm nhiều nhóm lồng nhau, phần code tổng hợp phải biết cách xử lý từng nhóm. Dùng Composite đưa công việc đó vào CompositeShow: mỗi nhóm tự lấy mô tả và tính thời lượng của các con. Client tiếp tục làm việc với Show như trước.

5.3 Không dùng kiểm tra số lượng thì sao

Nếu bỏ điều kiện trong constructor, chương trình vẫn có thể tạo CompositeShow với một show hoặc sáu show. Việc tính tổng có thể vẫn chạy nhưng đối tượng không còn đúng yêu cầu 2 đến 5 show con. Vì vậy, nhóm giữ kiểm tra số lượng ngay khi khởi tạo.

```
new CompositeShow(movie1); // Loi vi chi co 1 show
new CompositeShow(movie1, movie2); // Hop le vi co 2 show
```

Giới hạn được kiểm tra ở mỗi nhóm. Một sự kiện có thể chứa hơn 5 show đơn lẻ nếu các show được chia thành những nhóm hợp lệ; điều kiện không giới hạn tổng số show ở tất cả các tầng.

6 Nhận xét và kết luận của nhóm

6.1 Khi nào nên sử dụng Composite

Qua tình huống trên, nhóm chúng em thấy Composite phù hợp khi chương trình cần quản lý nhiều nhóm show, có nhóm lồng nhau và muốn dùng cùng một hàm cho show đơn lẻ lẫn show tổng hợp. Ví dụ, chức năng hiển thị thông tin có thể nhận một Movie, một suất chiếu hai phim hoặc cả sự kiện qua kiểu Show.

Nếu chỉ cộng thời lượng của vài show cố định một lần, cách cộng trực tiếp sẽ ngắn và dễ viết hơn. Composite có ích rõ hơn khi cần giữ cấu trúc nhóm và sử dụng lại các nhóm trong nhiều chương trình. Riêng bài tập này yêu cầu hai movie phải là một Show, nên CompositeShow đáp ứng đúng yêu cầu đó.

6.2 Những điểm cần lưu ý khi sử dụng code

Thứ nhất, một CompositeShow cần từ 2 đến 5 show con trực tiếp. Khởi tạo ngoài giới hạn này sẽ phát sinh IllegalArgumentException. Thứ hai, đoạn code chưa kiểm tra phần tử null, nên nếu truyền null thì khi lấy mô tả hoặc tính thời lượng có thể xảy ra NullPointerException.

Thứ ba, copy() hiện trả về null và được để lại cho Exercise 10. Vì vậy, ví dụ chỉ sử dụng description() và runningTime(), chưa sử dụng chức năng sao chép. Các số 250, 350 và 280 phút là kết quả tính theo dữ liệu minh họa và cách cộng thời lượng trong mã nguồn.

6.3 Kết luận

Nhóm chúng em sử dụng UML để minh họa cách các lớp phối hợp trong giải pháp. Movie và Concert là show đơn lẻ; CompositeShow chứa các Show con và cũng triển khai Show. Nhờ vậy, một nhóm show có thể được dùng ở nơi chương trình yêu cầu một Show.

Với sự kiện gồm một concert trước hai movie, nhóm tạo một CompositeShow cho suất chiếu hai phim rồi ghép suất chiếu với concert. Tổng thời lượng là 350 phút. So với cách tự cộng và ghép chuỗi ở client, giải pháp giữ được cấu trúc nhóm và giúp dùng lại hàm hiển thị thông tin mà không phải viết riêng cho từng trường hợp.

Tài liệu tham khảo: đề Exercise 1 và Solution to Exercise 1. Sơ đồ UML được nhóm xây dựng để trình bày rõ hơn cách hiểu giải pháp.